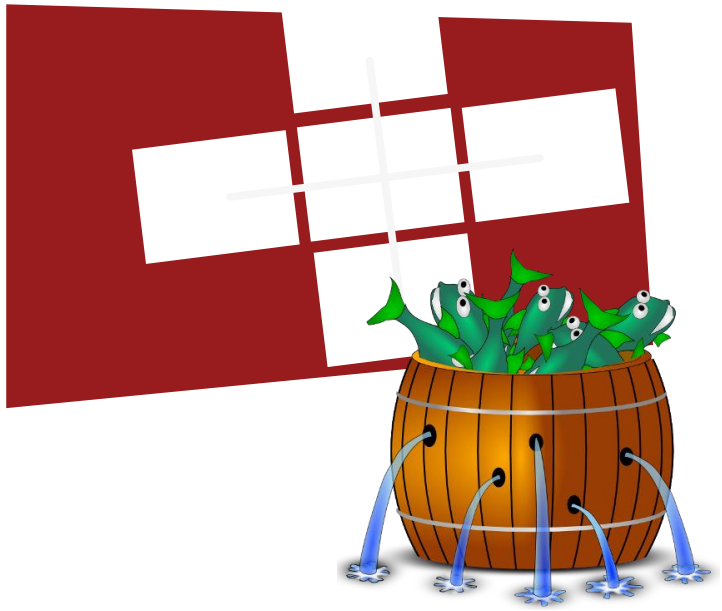
Mapping GitLab Attack Paths into BloodHound with OpenGraph

Marc André Tanner
IT Security Analyst
Compass Security

Marc André Tanner



https://github.com/martanne

Agenda

01 GitLab 101

02 Attack Paths

03 Graphing your own GitLab Instance

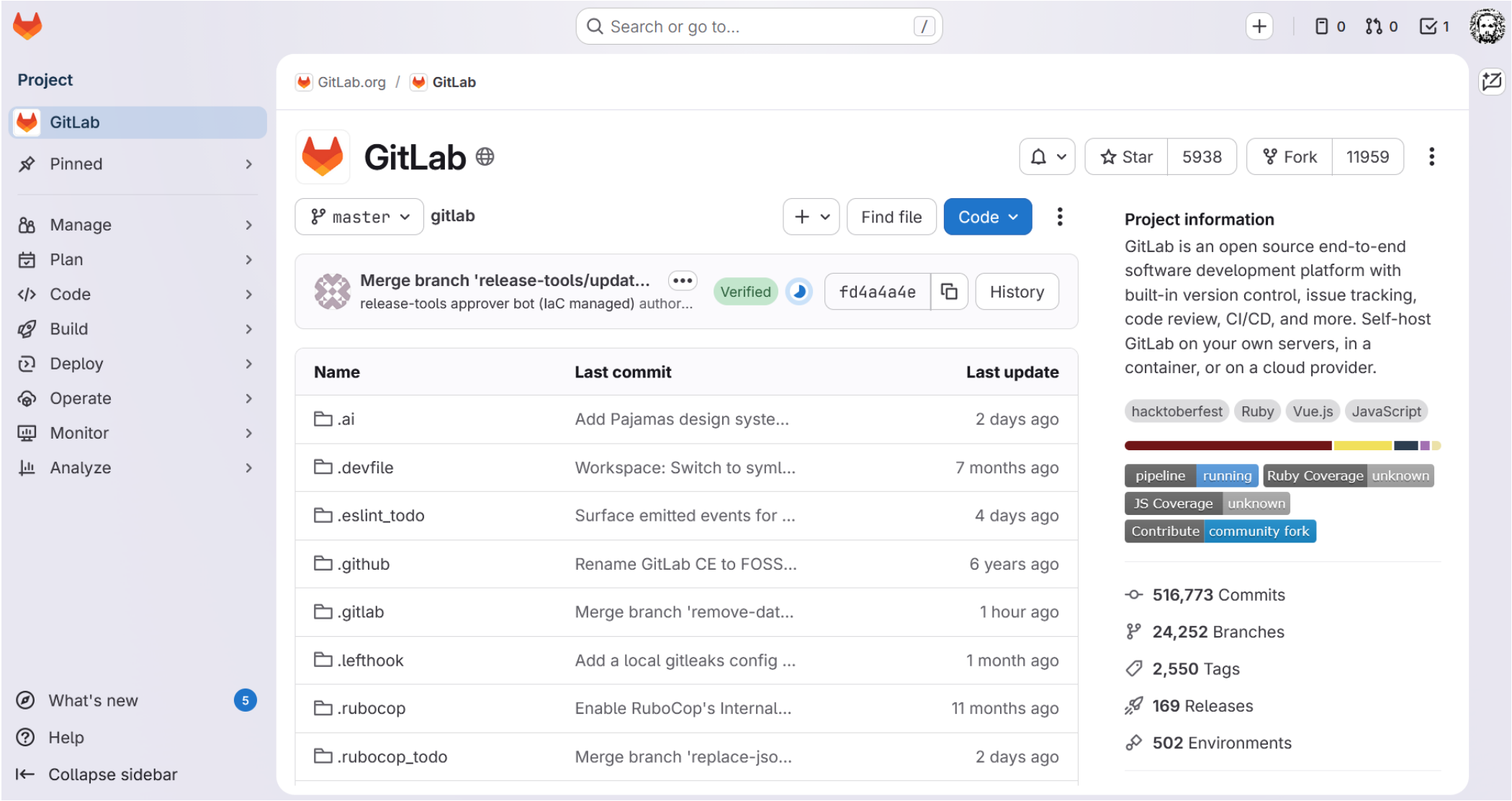
04 Defensive Considerations

05 Conclusion

GitLab 101

Permission model, authentication, CI/CD jobs

GitLab 101


Search or go to...
Project
GitLab
Pinned
Manage
Plan
Code
Build
Deploy
Operate
Monitor
Analyze
What's new
Help
Collapse sidebar
GitLab.org / GitLab
GitLab
Star 5938
Fork 11959
master
gitlab
Find file
Code
Merge branch 'release-tools/updat...
release-tools approver bot (IaC managed) author...
Verified
fd4a4a4e
History
Name
Last commit
Last update
.ai
Add Pajamas design syste...
2 days ago
.devfile
Workspace: Switch to syml...
7 months ago
.eslint_todo
Surface emitted events for ...
4 days ago
.github
Rename GitLab CE to FOSS...
6 years ago
.gitlab
Merge branch 'remove-dat...
1 hour ago
.lefthook
Add a local gitleaks config ...
1 month ago
.rubocop
Enable RuboCop's Internal...
11 months ago
.rubocop_todo
Merge branch 'replace-jso...
2 days ago
Project information
GitLab is an open source end-to-end software development platform with built-in version control, issue tracking, code review, CI/CD, and more. Self-host GitLab on your own servers, in a container, or on a cloud provider.
hacktoberfest
Ruby
Vue.js
JavaScript
pipeline running
Ruby Coverage unknown
JS Coverage unknown
Contribute community fork
516,773 Commits
24,252 Branches
2,550 Tags
169 Releases
502 Environments

GitLab 101

Introduction

- Community edition: open source (MIT license), self-managed
- Enterprise edition: source available (proprietary)
  - Deployment: **Self-managed,** GitLab.com SaaS, Dedicated
  - Feature tiers: Free, Premium, Ultimate
- REST & GraphQL APIs available
- “Modern DevOps platform”

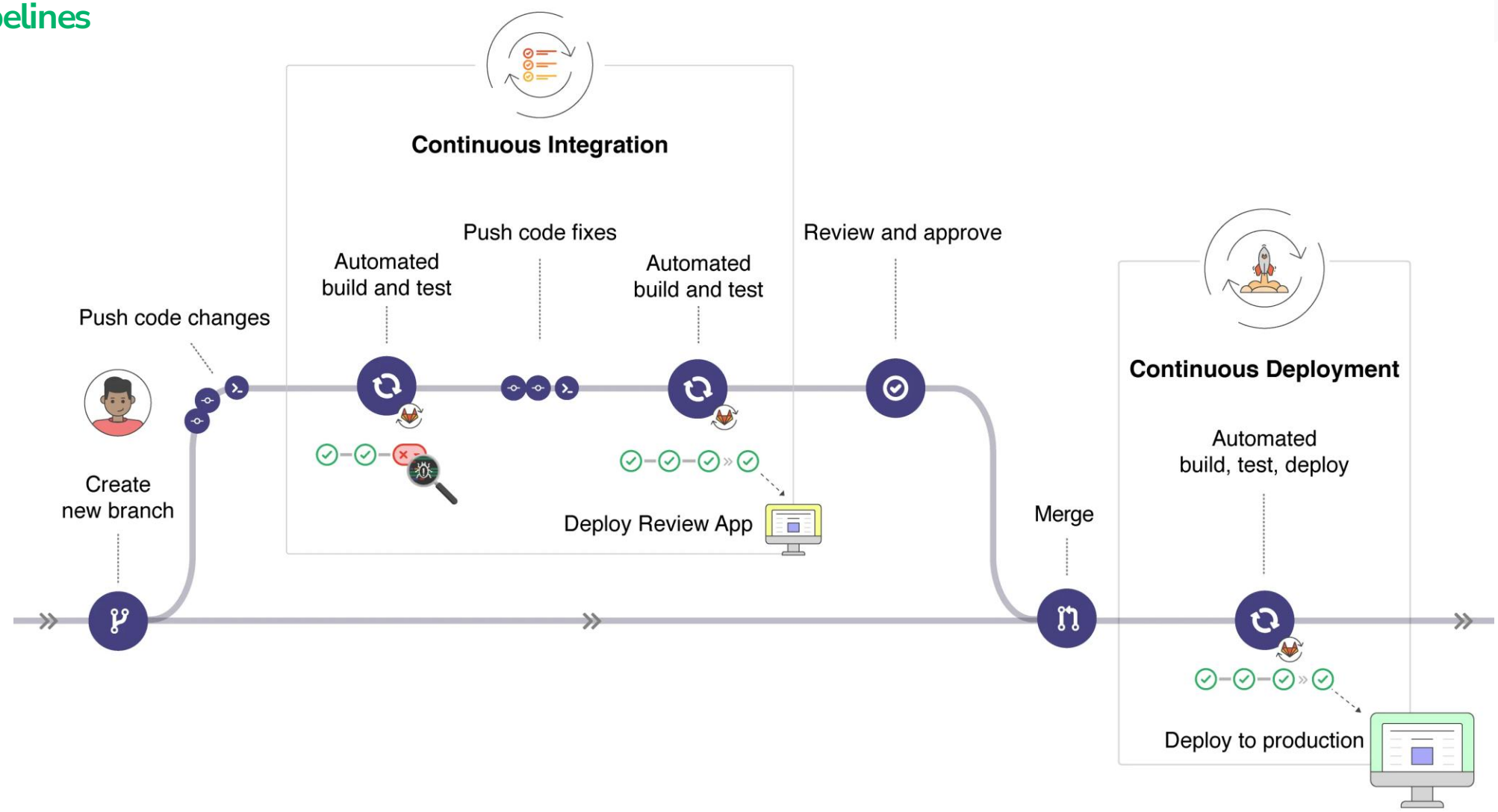
GIT.PWN.GIFT

Object Information

Node Type:	GLInstance
Object ID:	GIT.PWN.GIFT
Can Create Group:	TRUE
Default Group Visibility:	private
Default Project Visibility:	private
Disable Personal Access Tokens:	FALSE
Domain Denylist Enabled:	FALSE
Email Confirmation Setting:	off
Enterprise:	TRUE
Revision:	91256b9c8dc
Runners:	2
Shared Runners Enabled:	TRUE
Signup Enabled:	TRUE
Users:	11
Version:	18.9.1-ee

GitLab 101

CI/CD Pipelines



Attack avenues

Read

- Read intellectual property
- Leaked credentials in
  - Repositories
  - Job logs
  - Artifacts
- Configured secrets in
  - CI/CD variables
  - Secure files

Write

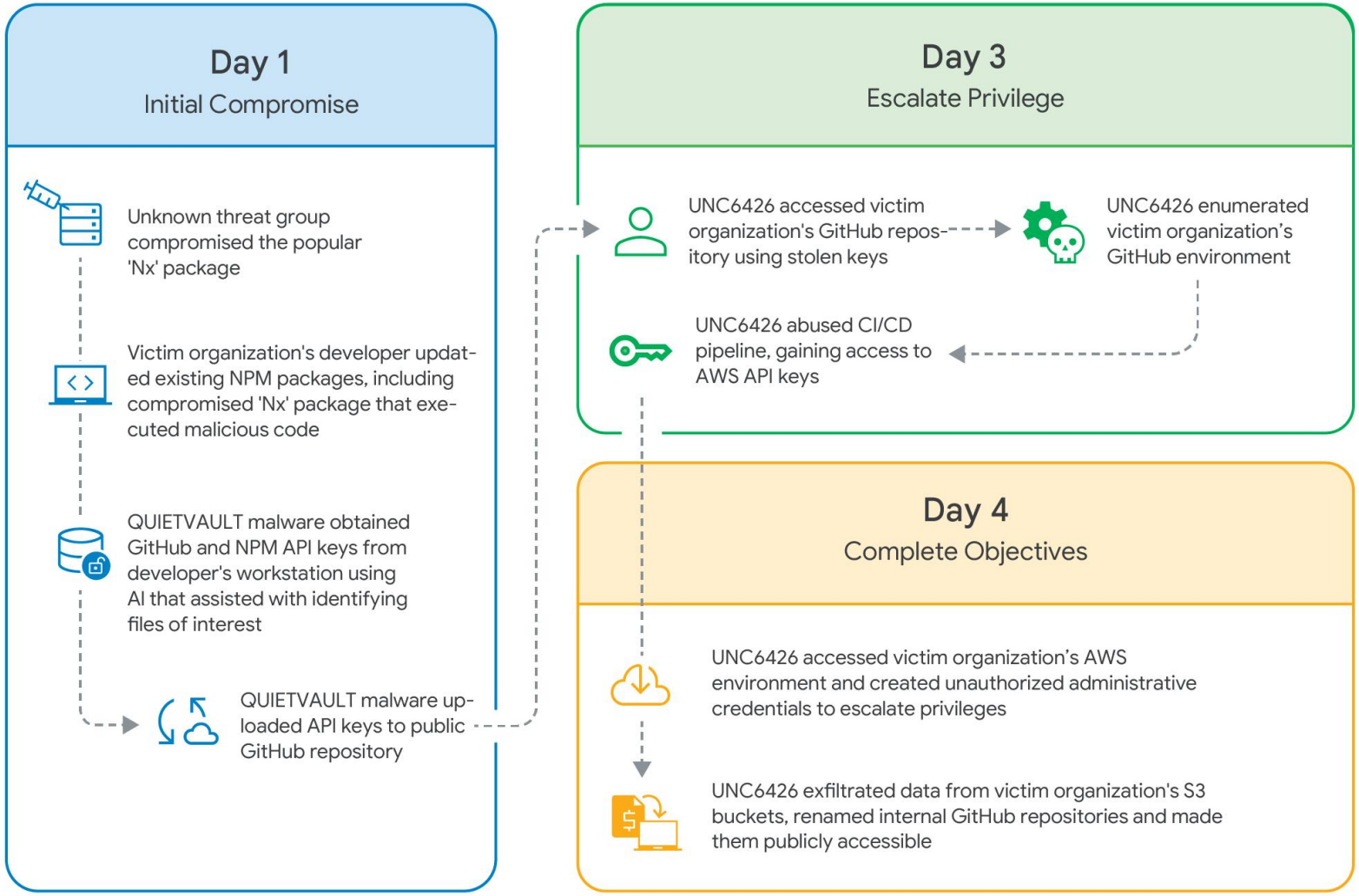
- Supply chain attacks
- Bot abuse
- Register malicious runner

Execute

- Access connected systems
- Shared runners
- Abuse trust relationships, e.g., federated identities

Real-world attack paths

UNC6423, H2 2025



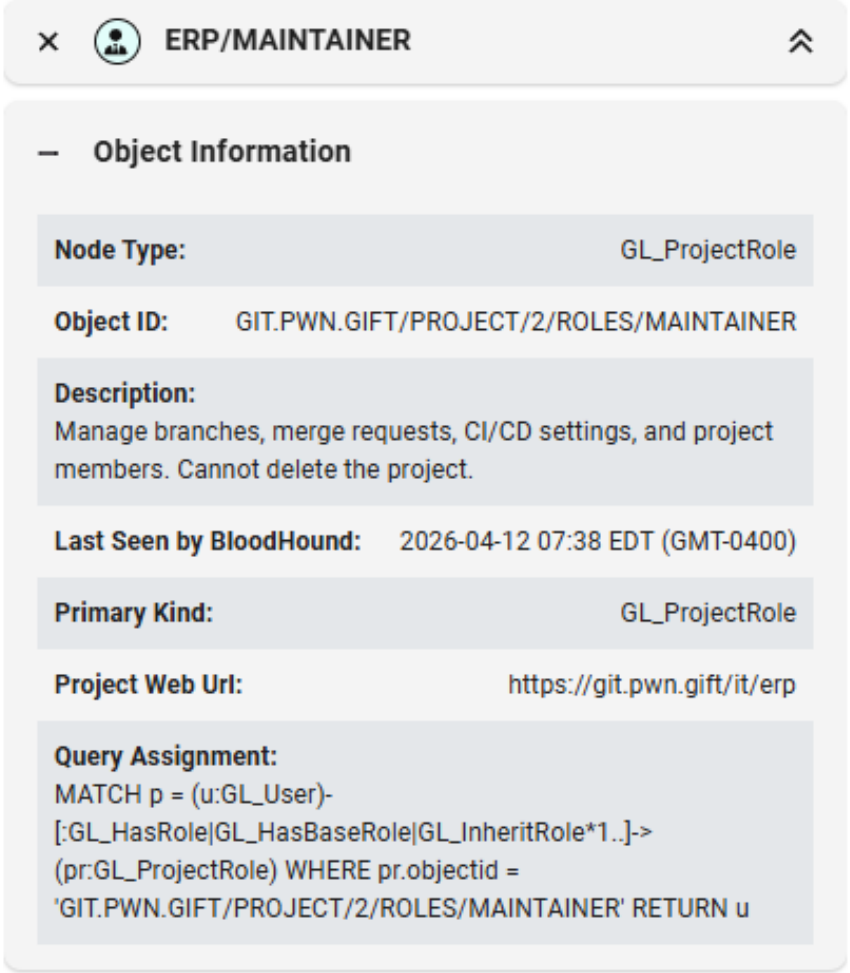
https://cloud.google.com/security/report/resources/cloud-threat-horizons-report-h1-2026

GitLab permission model

Project roles

Permissions are cumulative*

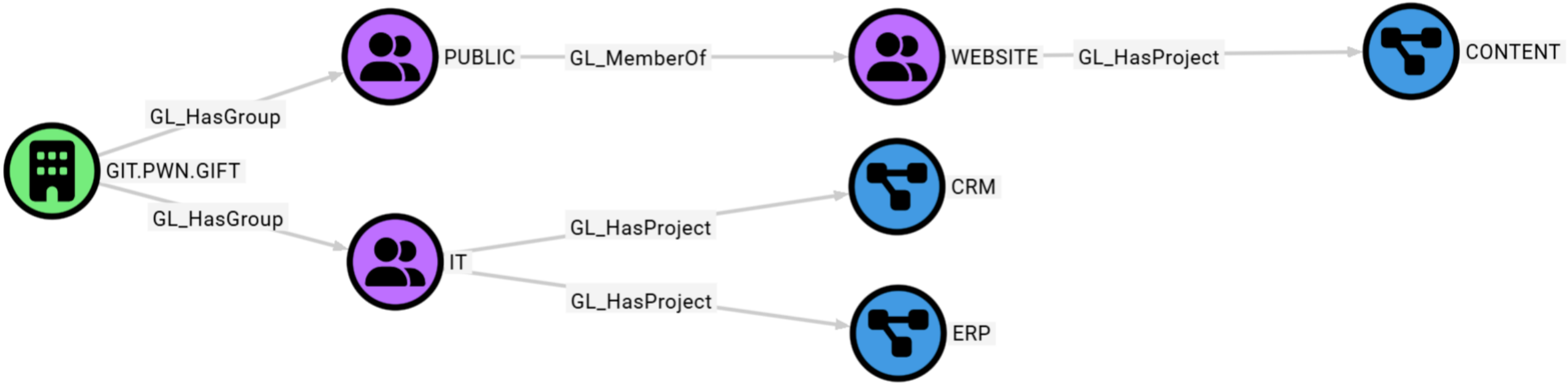
- Owner
- Maintainer
- Developer
- …
- Guest



GitLab permission model

Group roles

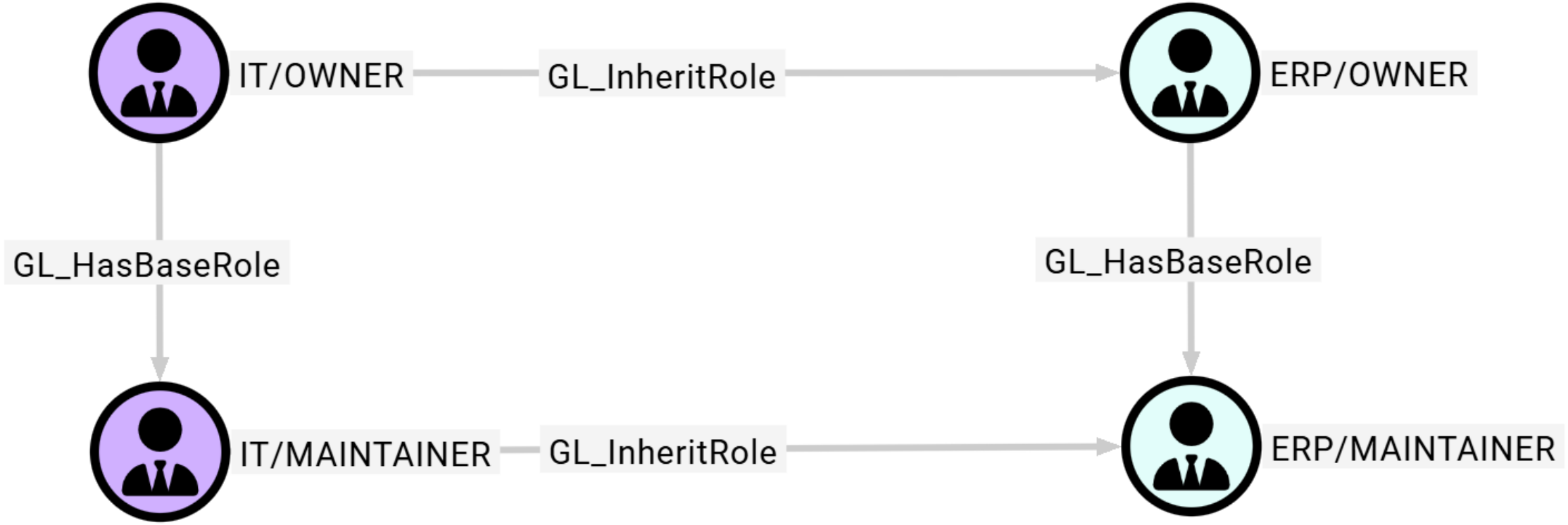
- Hierarchical groups
- Members are assigned a role
- Permissions are inherited down



GitLab permission model

Group roles

- Hierarchical groups
- Members are assigned a role
- Permissions are inherited down

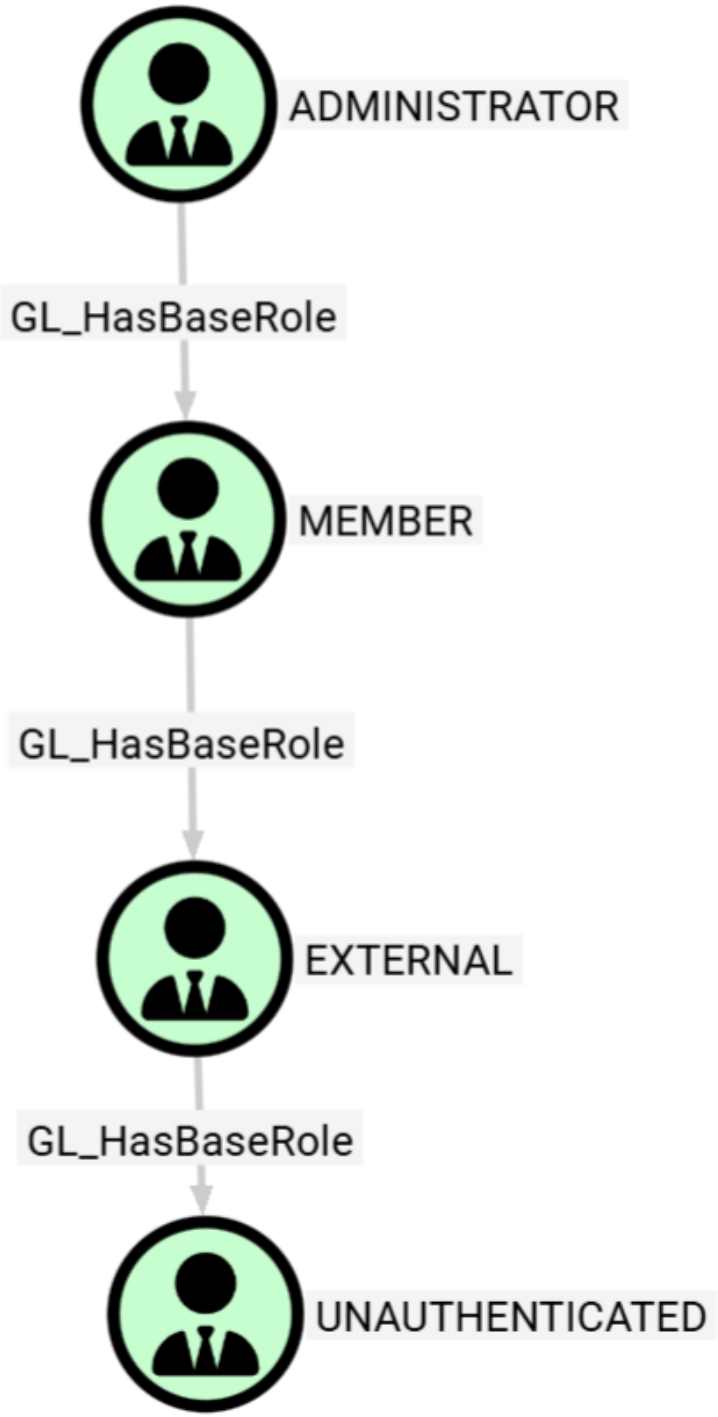


GitLab permission model

Instance roles

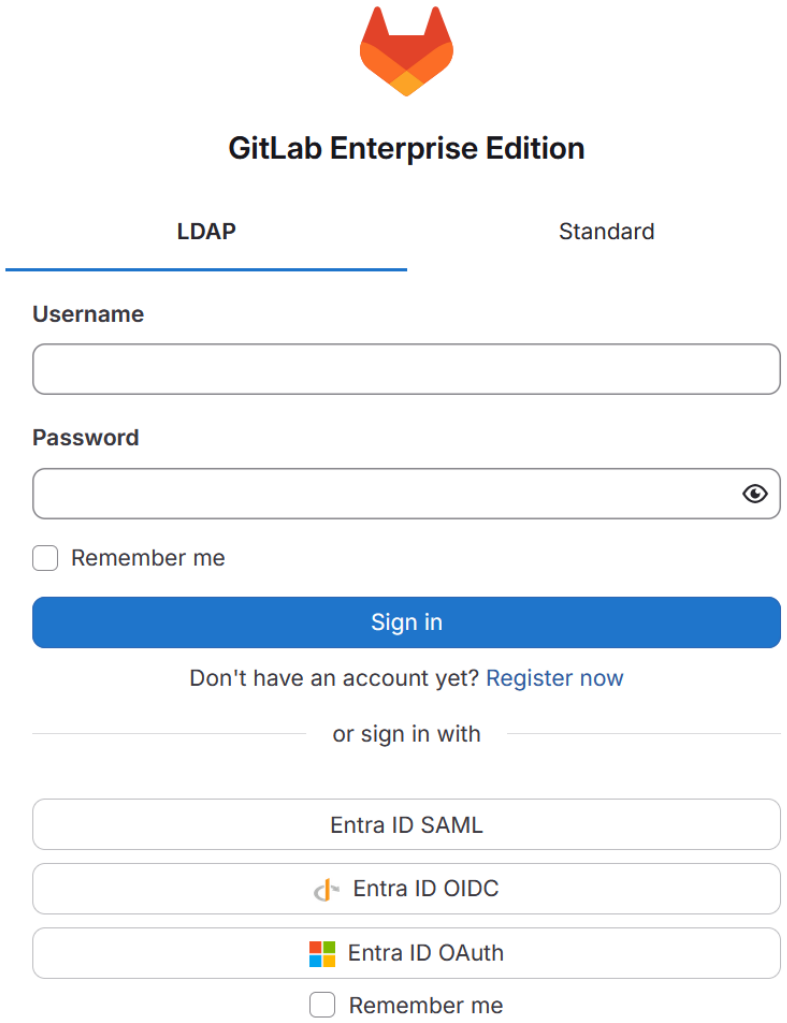
Used to model instance wide permissions

- Administrator
- Member
- External
- Unauthenticated



GitLab authentication

LDAP, OAuth, OIDC, SAML, ...

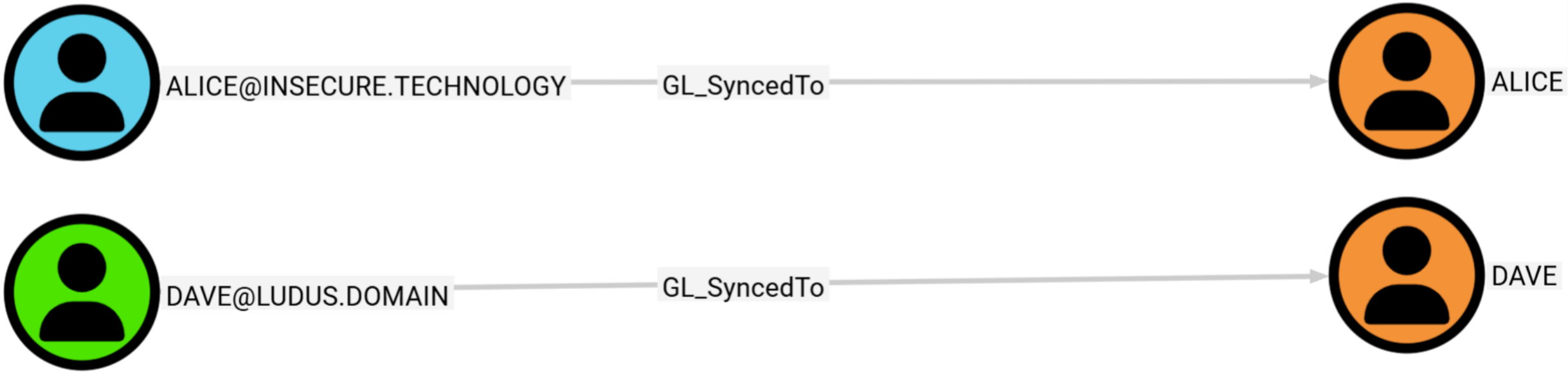


GitLab authentication

Hybrid edges

GL_SyncedTo

- Entra ID: SAML, OAuth, OIDC
- Active Directory: LDAP



Attack Paths

Initial access, privilege escalation and lateral movement

Initial access

Getting the first token

Possible initial access vectors

- Compromise linked identity (Active Directory, Entra ID)
- Create your own account, if missing sign-up restrictions without admin approval
- Device code phishing
  - https://github.com/Maldev-Academy/GitLabDeviceCodePhishing/
- Secret exposure in public resource
  - Repository
  - Job logs
  - Artifacts

Initial access

Secrets in repository history

```
$ git log -p

commit 99e2507e66cd00f7f1ec79c441d3730c8e7dcfc6 (HEAD -> main, origin/main)
Author: Alice <alice@git.pwn.gift>
Date:   Thu Mar 26 13:54:35 2026 +0100

    Update README

diff --git a/README.md b/README.md
index e769d20..514ac13 100644
--- a/README.md
+++ b/README.md
@@ -1,5 +1,5 @@
 This repository holds my configuration files.

 ```
-git clone https://oauth2:glpat-GJyreGz-XoceH12iOUZI9W86MQp1OjIH@git.pwn.gift/alice/config.git
+git clone https://git.pwn.gift/alice/config.git
 ```
```

https://github.com/trufflesecurity/trufflehog

Initial access

Secrets in job logs

Definition

```
$ cat .gitlab-ci.yml
build:
  script:
    - set -x
    - echo "$SSH_KEY" > id_ed25519
    - chmod 600 id_ed25519
    - export GIT_SSH_COMMAND="ssh -i id_ed25519 -o StrictHostKeyChecking=no"
    - git clone git@git.pwn.gift:alice/config.git
```

Execution

```
$ set -x
$ echo "$SSH_KEY" > id_ed25519
+ echo '$ echo "$SSH_KEY" > id_ed25519'
+ echo '-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACBt0EyayyzR/TrpSOZmsmHezYmmyiMcoWDXk+V47ML050AAAJDy2ggH79oI
```


https://blog.compass-security.com/2024/12/harvesting-gitlab-pipeline-secrets/

Initial access

Secrets in job artifacts

Definition

```
$ cat .gitlab-ci.yml

build:
  script:
    - echo $GITLAB_TOKEN | tee token.txt
  artifacts:
    paths:
      - token.txt
```

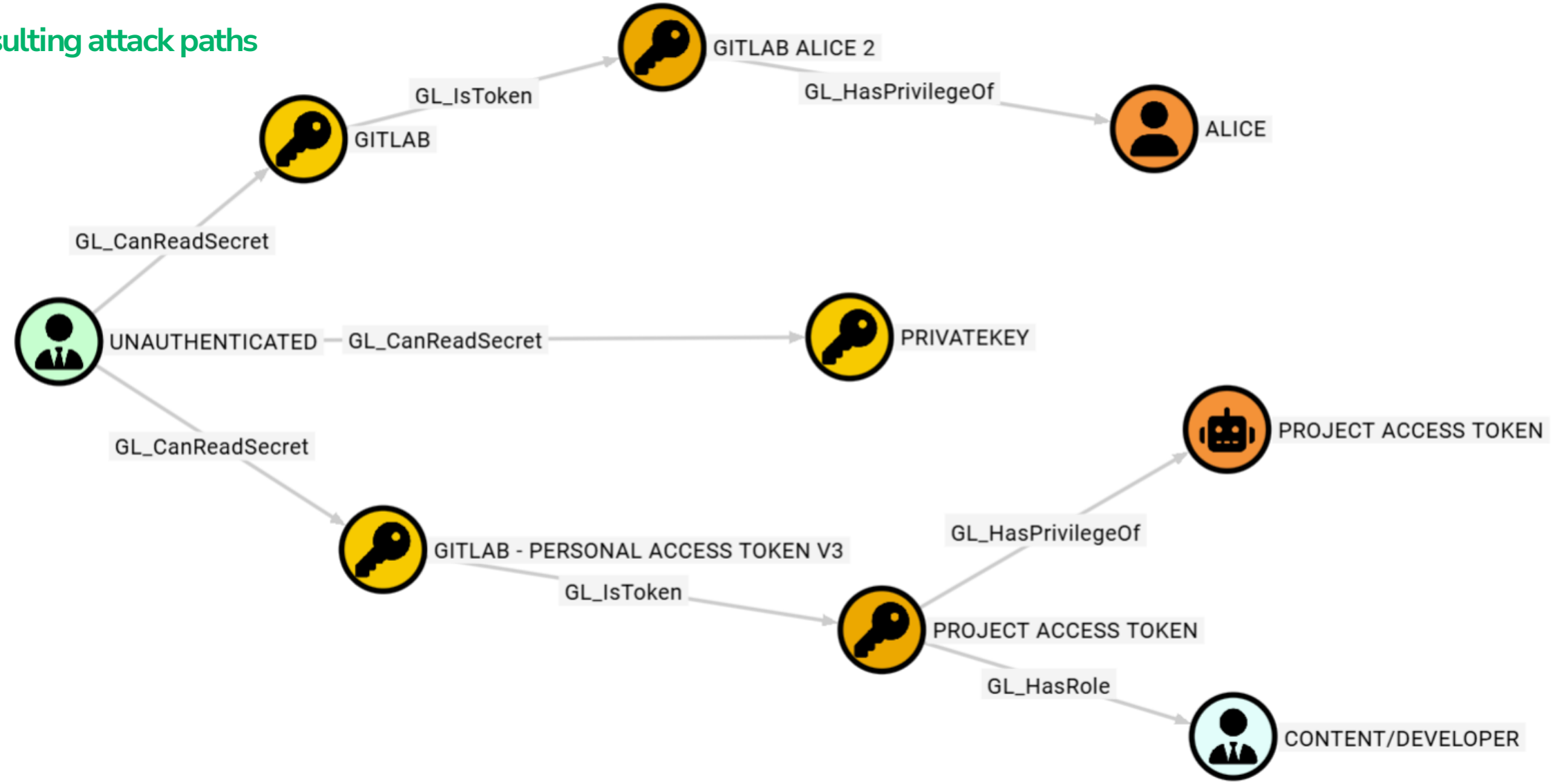
Execution

```
$ echo $GITLAB_TOKEN | tee token.txt
glpat-[MASKED]
Uploading artifacts for successful job    00:00
Uploading artifacts...
token.txt: found 1 matching artifact files and
directories
```

https://blog.compass-security.com/2024/12/harvesting-gitlab-pipeline-secrets/

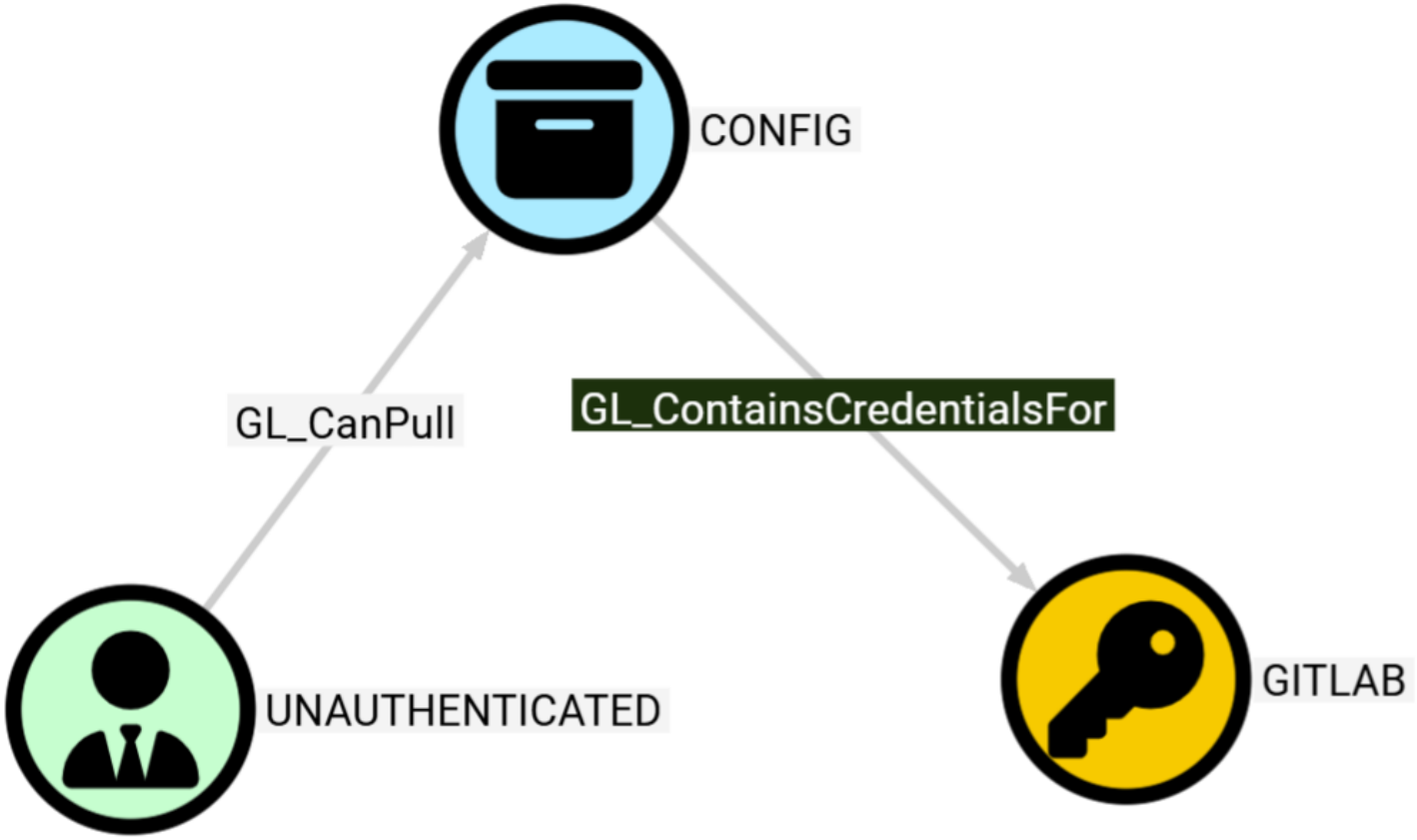
Initial access

Resulting attack paths



Initial access

Secrets in repository history



GL_ContainsCredentialsFor

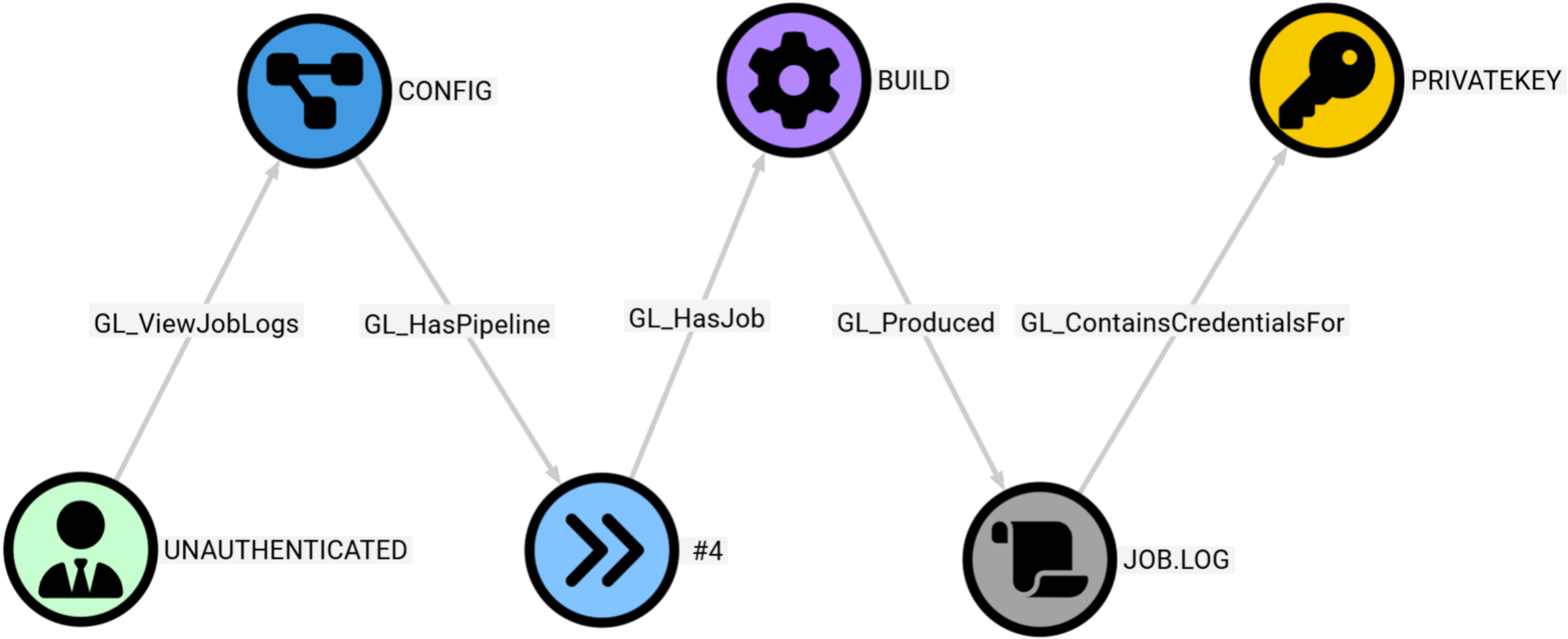
Relationship Information

Source Node:	CONFIG
Target Node:	GITLAB
Commit:	21080754b8d06a98fdcb9f6cbd4efc31ed782e5f
Detector:	Gitlab
File:	README.md
Last Seen by BloodHound:	2026-04-12 15:38 EDT (GMT-0400)
Line:	4
Link:	https://git.pwn.gift/alice/config/blob/21080754b8d06a98fdc...

The edge **GL_ContainsCredentialsFor** does not have any additional contextual information at this time.

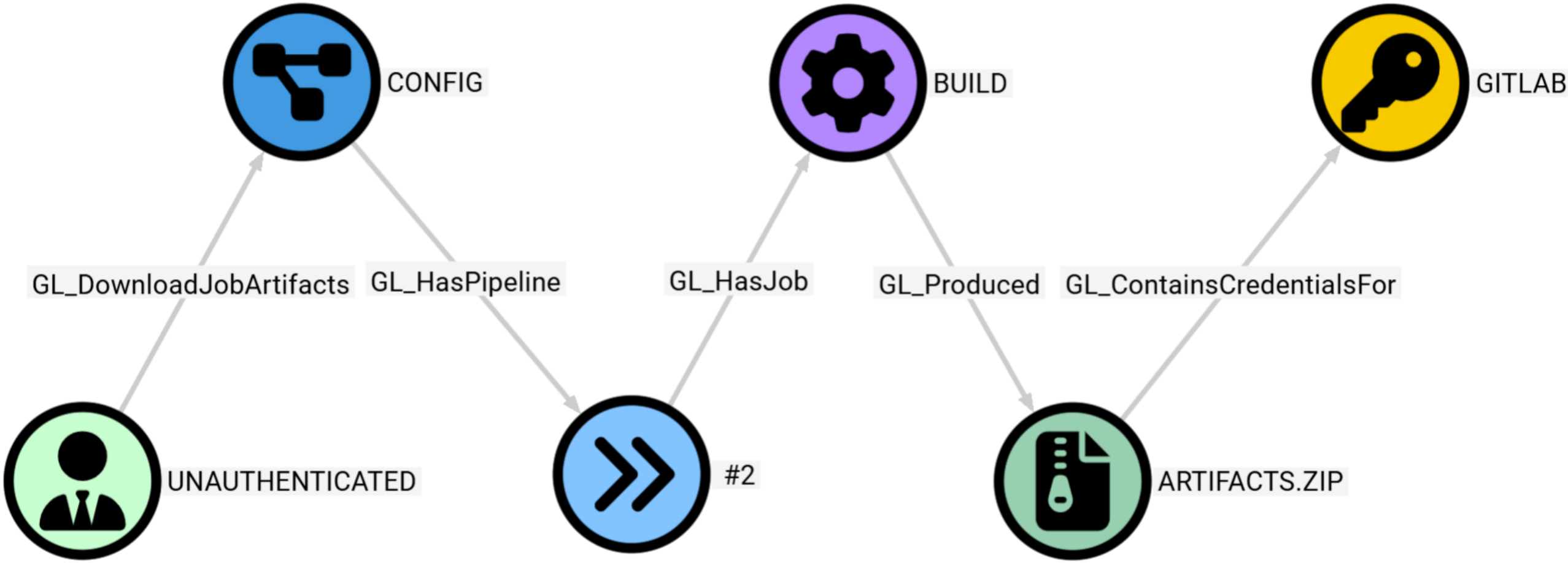
Initial access

Secrets in job logs



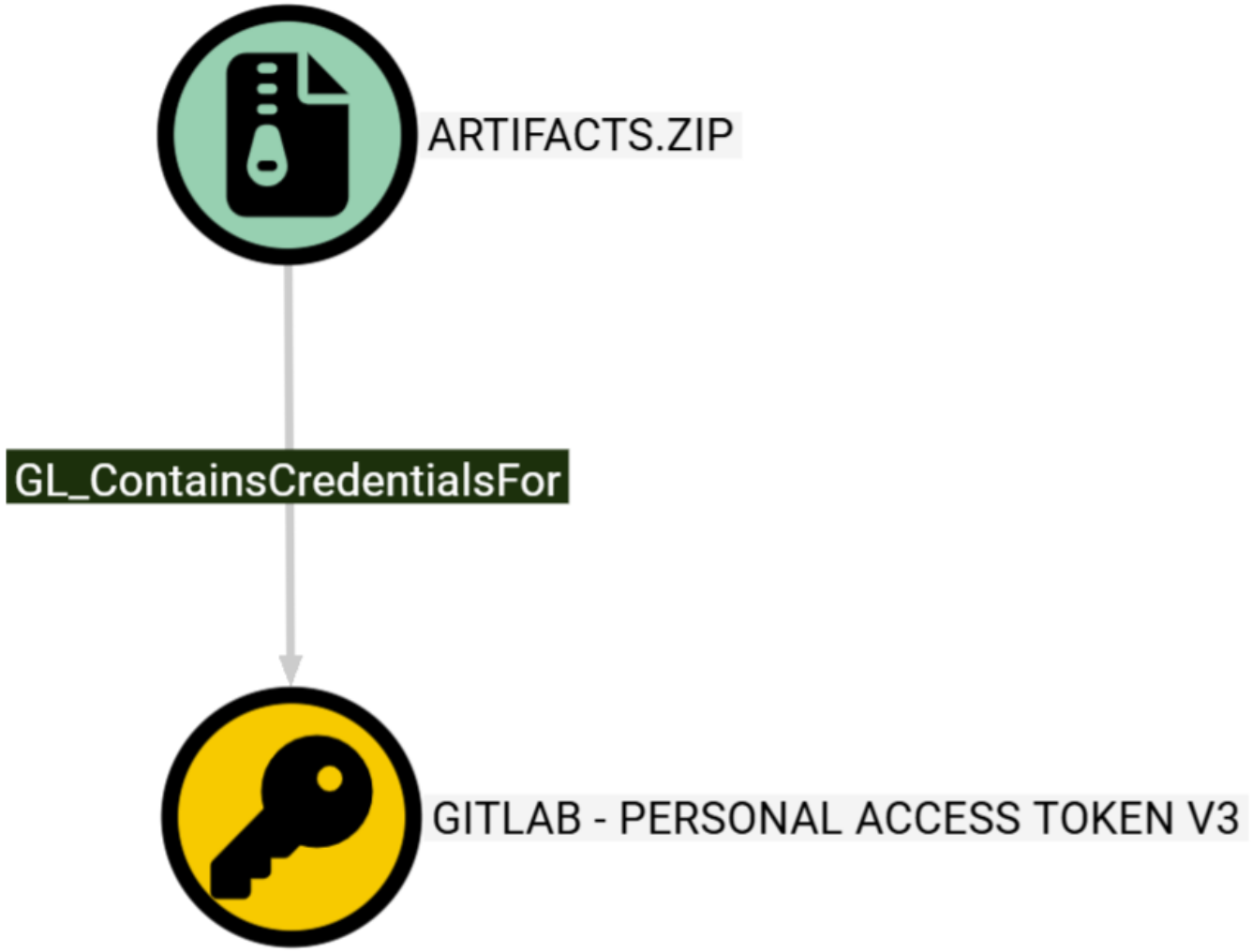
Initial access

Secrets in job artifacts



Initial access

Secrets in job artifacts



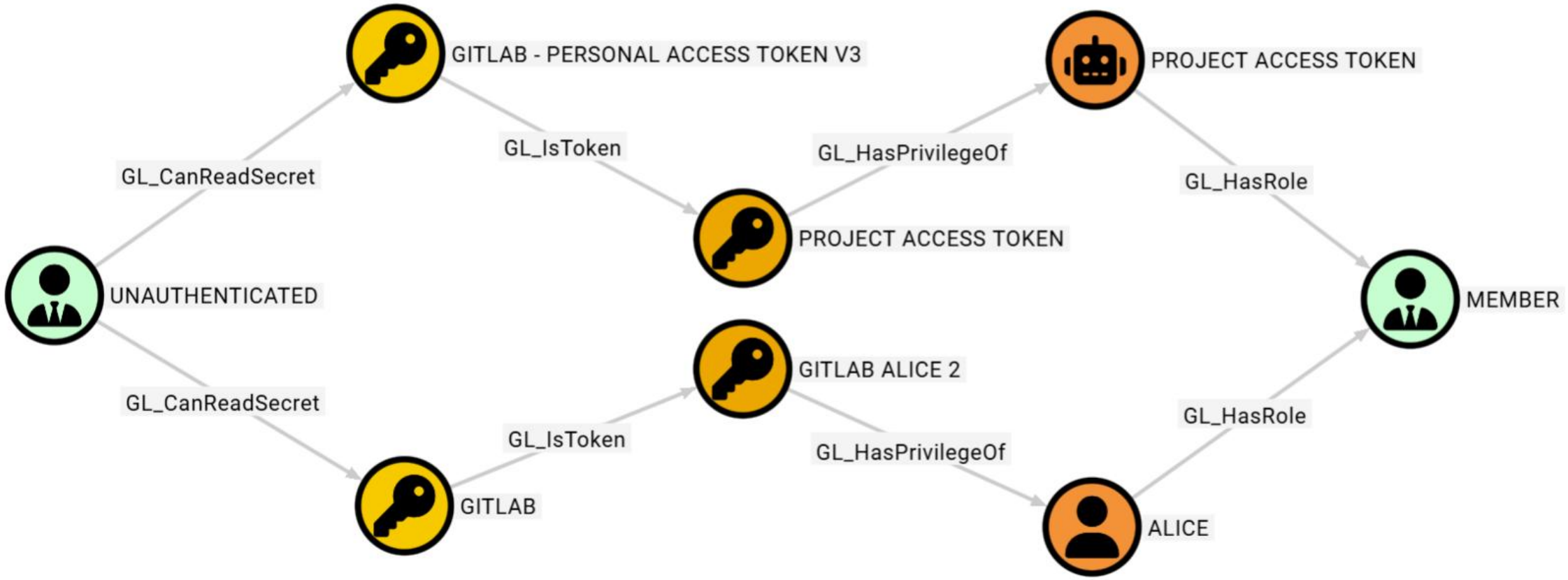
GL_ContainsCredentialsFor

Relationship Information

Source Node:	ARTIFACTS.ZIP
Target Node:	GITLAB - PERSONAL ACCESS TOKEN V3
Confidence:	high
File:	token.txt
Job Name:	build
Last Seen by BloodHound:	2026-04-12 15:38 EDT (GMT-0400)
Rule:	Gitlab - Personal Access Token v3
Source Type:	archive

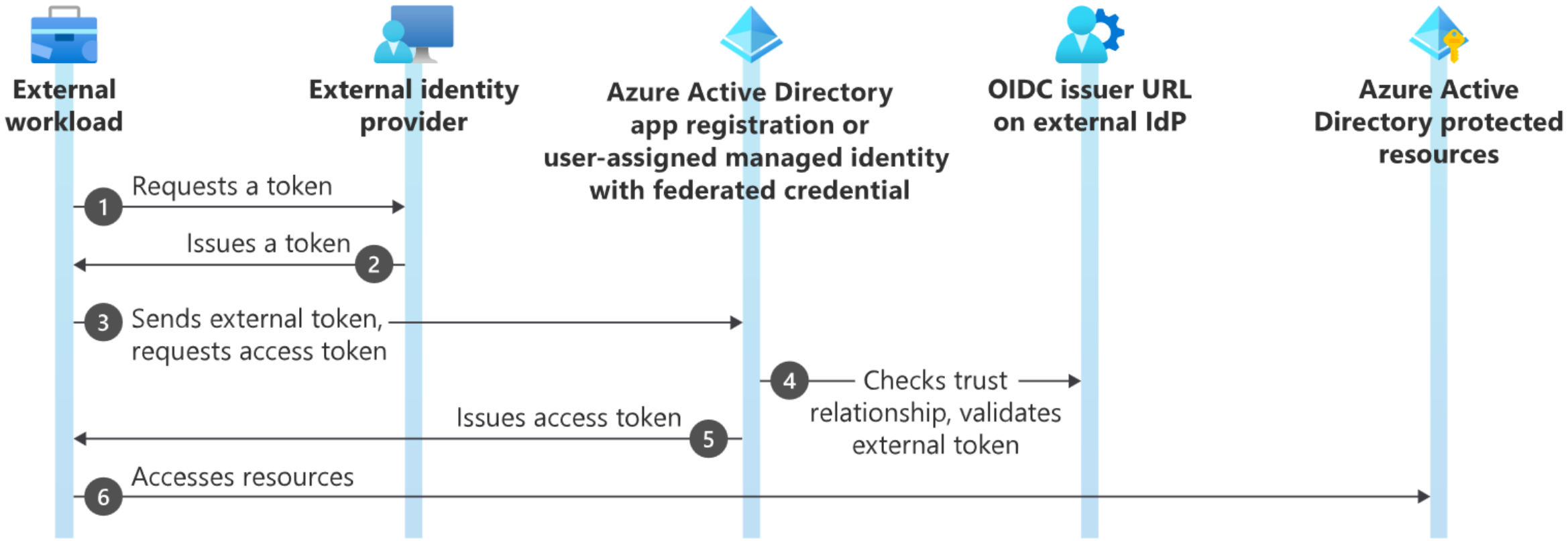
The edge **GL_ContainsCredentialsFor** does not have any additional contextual information at this time.

Initial access



Workload identity federation

Authentication without secrets



https://learn.microsoft.com/en-us/entra/workload-id/workload-identity-federation

Abusing OIDC trust relationships

Entra ID managed identity with registered federated credential

Microsoft Azure | Search resources, services, and docs (G+/) | Copilot

Home > Managed Identities > gitlab-managed-identity

gitlab-managed-identity | Federated credentials

Managed Identity

Federated credentials

Configure an identity from an external OpenID Connect Provider to get tokens as this managed identity to access Microsoft Entra ID protected services.Learn more about how to create an identity from an external OpenID

+ Add Credential

1 of 20 configured

Name ↑	Issuer	Subject Identifier	Delete
gitlab-federated	https://git.pwn.gift	project_path:pub/www/content:ref_type:branch:ref:main	

https://learn.microsoft.com/en-us/entra/workload-id/workload-identity-federation-create-trust-user-assigned-managed-identity

Abusing OIDC trust relationships

GitLab issues OIDC ID tokens for CI/CD jobs

```
$ cat .gitlab-ci.yml

default:
  image: mcr.microsoft.com/azure-cli:latest

variables:
  AZURE_CLIENT_ID: "8a2a6ebf-d292-4bee-9e73-bdf4dc425d52"
  AZURE_TENANT_ID: "925c2cd8-a177-41a5-93f3-1257ffd75334"

auth:
  id_tokens:
    GITLAB_OIDC_TOKEN:
      aud: api://AzureADTokenExchange
  script:
    - az login --service-principal -u $AZURE_CLIENT_ID -t $AZURE_TENANT_ID --federated-token
       $GITLAB_OIDC_TOKEN
```

https://docs.gitlab.com/ci/cloud_services/azure/#retrieve-a-temporary-credential

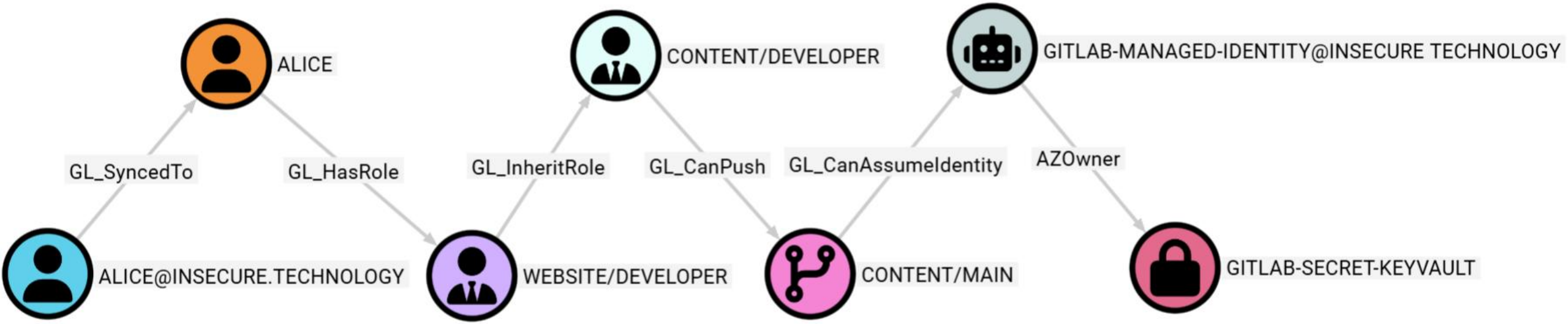
Abusing OIDC trust relationships

CI/CD job execution

```
$ az login --service-principal -u $AZURE_CLIENT_ID -t $AZURE_TENANT_ID --federated-token $GITLAB_OIDC_TOKEN
[
  {
    "cloudName": "AzureCloud",
    "homeTenantId": "925c2cd8-a177-41a5-93f3-1257ffd75334",
    "id": "b8f0fe9a-a35f-42a0-8035-fc09bb109e79",
    "isDefault": true,
    "managedByTenants": [],
    "name": "SUB-PRIMARY",
    "state": "Enabled",
    "tenantId": "925c2cd8-a177-41a5-93f3-1257ffd75334",
    "user": {
      "name": "8a2a6ebf-d292-4bee-9e73-bdf4dc425d52",
      "type": "servicePrincipal"
    }
  }
]
```

Abusing OIDC trust relationships

Resulting attack path



Runner abuse

Poisoned pipelines

- CI/CD jobs are executed by (instance/group/project) runners
- Self-hosted (shared) runners are rarely a good idea
  - Privileged Docker containers
  - Domain-joined Windows runners
- Abuse potential depends on configured executor

IT / ERP / Jobs / #8

build

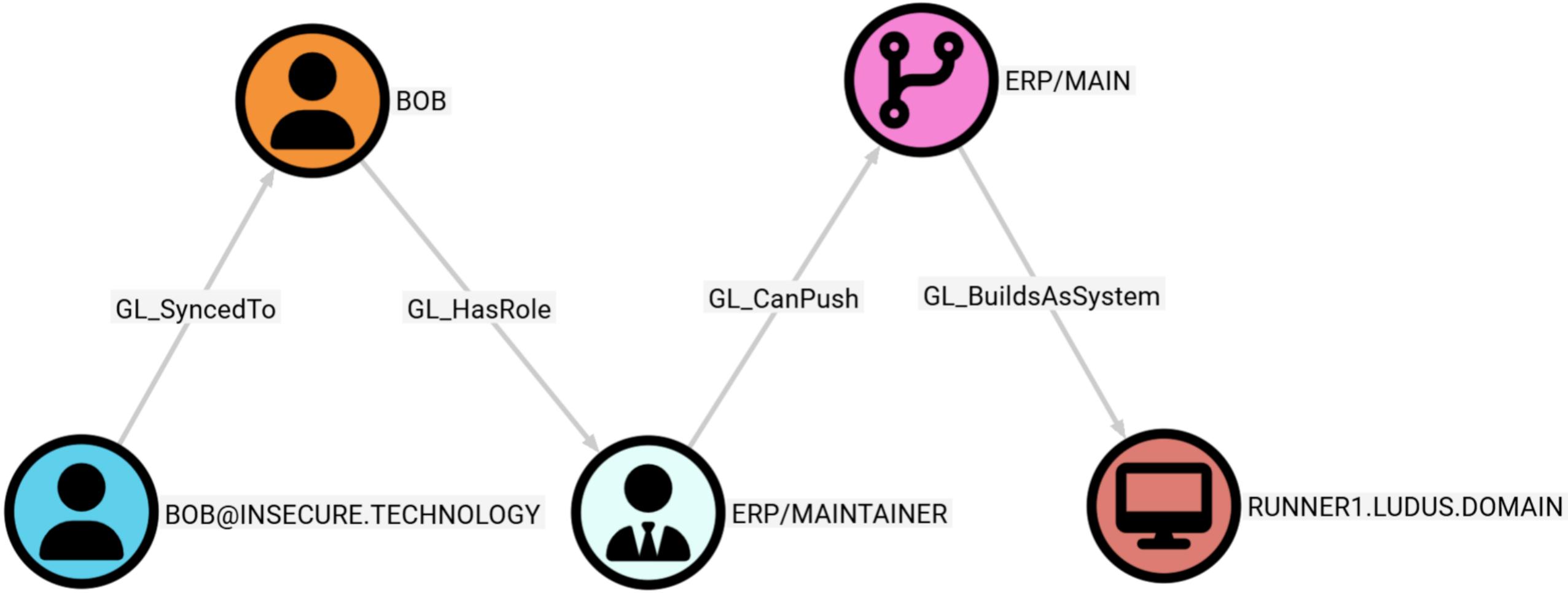
Passed Started 1 day ago by Bob

Log timestamps in UTC.

```
20:34:41 Running with gitlab-runner 18.9.0 (07e534ba)
20:34:41   on runner1 YkIEFE1pW, system ID: s_6805e459598d
20:34:41 Preparing the "shell" executor
20:34:41 Using Shell (powershell) executor...
20:34:41 Preparing environment
20:34:41 Running on RUNNER1...
20:34:41 Getting source from Git repository
20:34:43 Executing "step_script" stage of the job script
20:34:44 $ whoami /all
20:34:44 USER INFORMATION
20:34:44 ----------------
20:34:44 User Name           SID
20:34:44 =================== ========
20:34:44 nt authority\system S-1-5-18
```

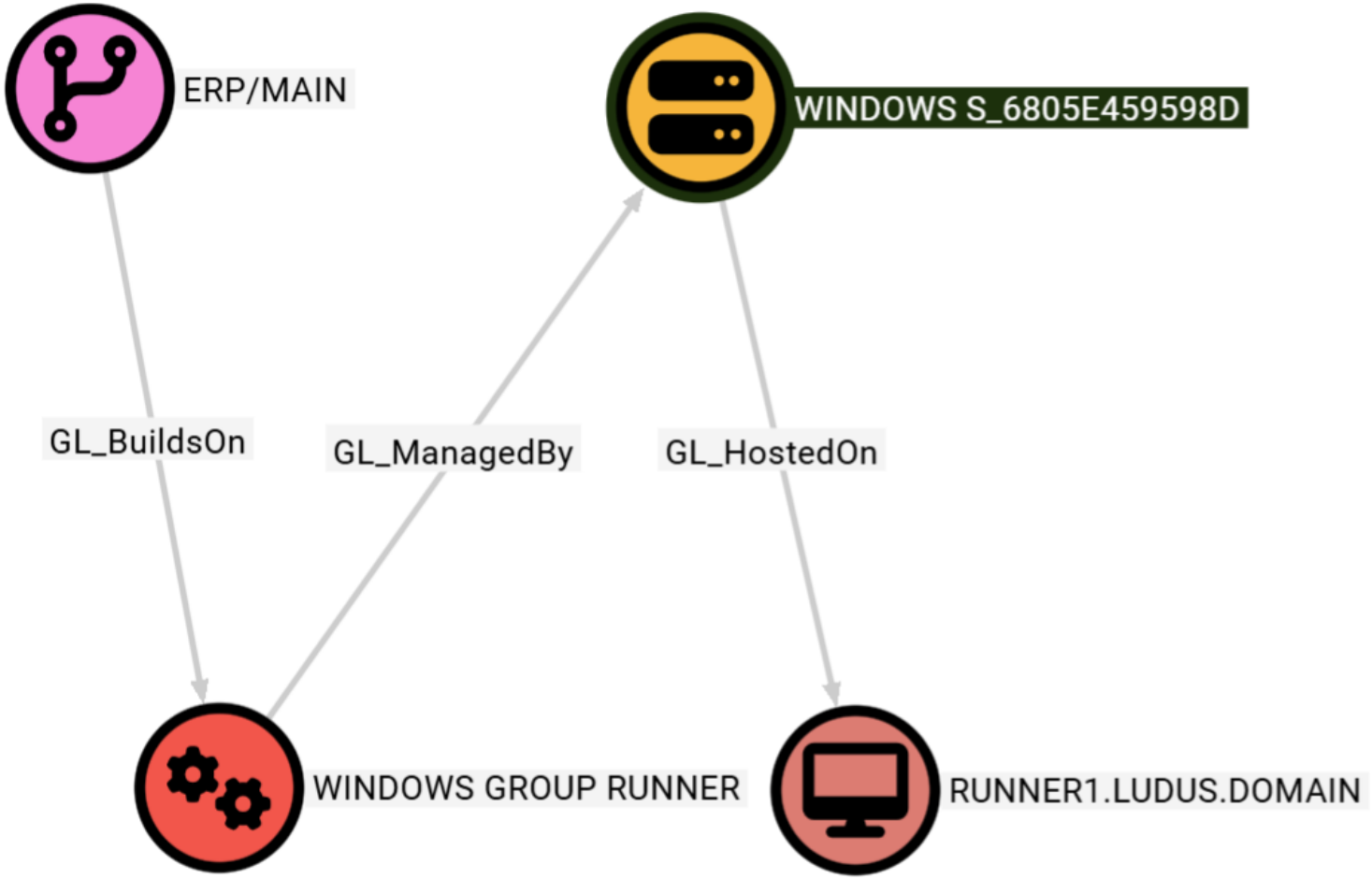
Runner abuse

Resulting attack path for domain-joined Windows runners



Runner abuse

Resulting attack path for domain-joined Windows runners



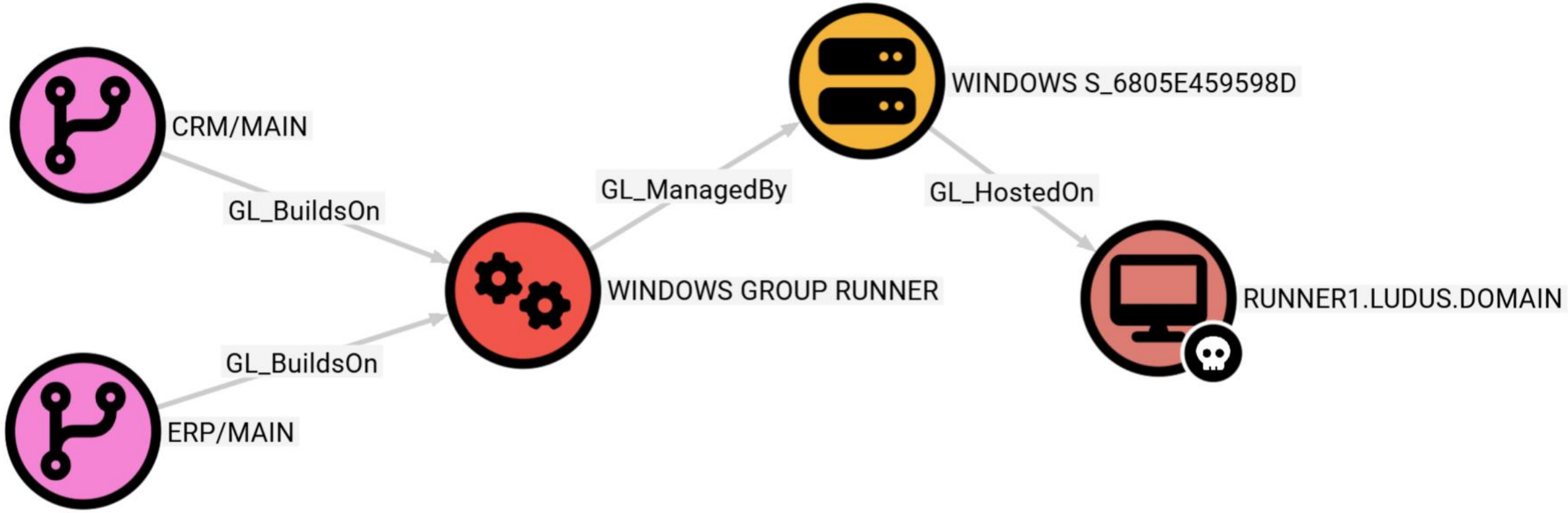
WINDOWS S_6805E459598D

Object Information

Node Type:	GL_RunnerManager
Object ID:	GIT.PWN.GIFT/RUNNER-MANAGER/1
Architecture:	amd64
Contacted At:	2026-04-08T19:02:40Z
Created At:	2026-02-25T15:23:37Z
Executor:	shell
Ip Address:	10.10.10.13
Job Exec Status:	IDLE
Last Seen by BloodHound:	2026-04-12 07:38 EDT (GMT-0400)
Platform:	windows
Primary Kind:	GL_RunnerManager

Tamper projects being built on same runner

Supply chain attack



Renovate update bot

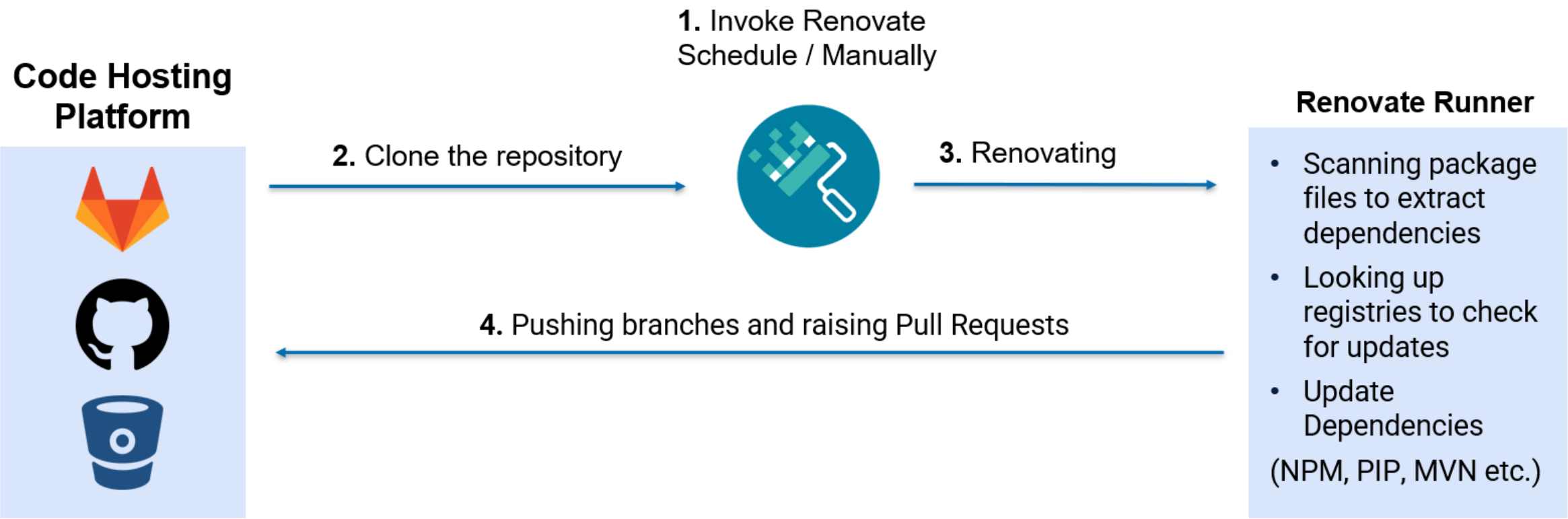
Privilege escalation opportunities

- Renovate bot to automatically update software dependencies
- Usually periodically ran via pipeline schedules
- Can be configured to auto discover projects
- Uses the native package manager to check for updates (code execution assumed)
- Creates merge requests for outdated components
  - Needs at least developer role
- Supports auto merge functionality
  - Requires merge permissions

https://docs.renovatebot.com/

Renovate update bot

Renovate bot abuse



https://blog.compass-security.com/2025/05/renovate-keeping-your-updates-secure/

> With GitLab's current security model, we find the risks of running a *public* bot service like Renovate are too high. Therefore we stopped hosting Renovate on GitLab, and are waiting for a better security model.

– **Renovate Documentation**

https://docs.renovatebot.com/gitlab-bot-security/#risks-of-hosting-a-renovate-gitlab-appbotservice

Renovate update bot abuse

Vulnerable configuration

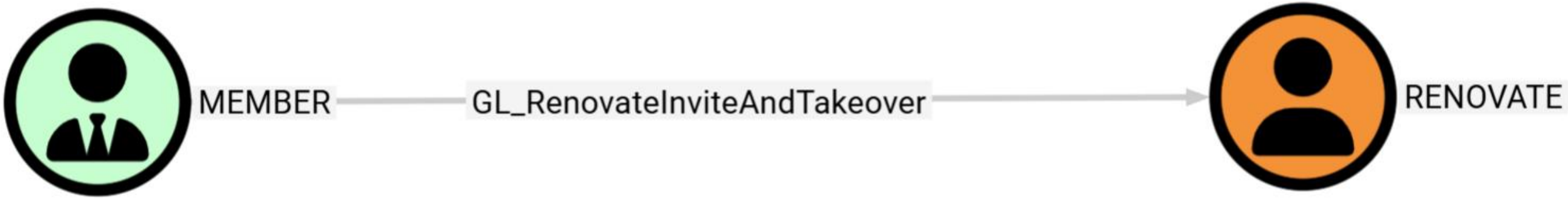
```
$ cat .gitlab-ci.yml

update-dependencies:
  image: renovate/renovate:latest
  rules:
    - if: '$CI_PIPELINE_SOURCE == "schedule"'
    - if: '$CI_PIPELINE_SOURCE == "web"'
  variables:
    RENOVATE_PLATFORM: gitlab
    RENOVATE_ENDPOINT: $CI_SERVER_URL/api/v4
    RENOVATE_TOKEN: $RENOVATE_TOKEN
    RENOVATE_AUTODISCOVER: true
  script:
    - renovate
```

https://docs.renovatebot.com/modules/platform/gitlab/

Renovate update bot abuse

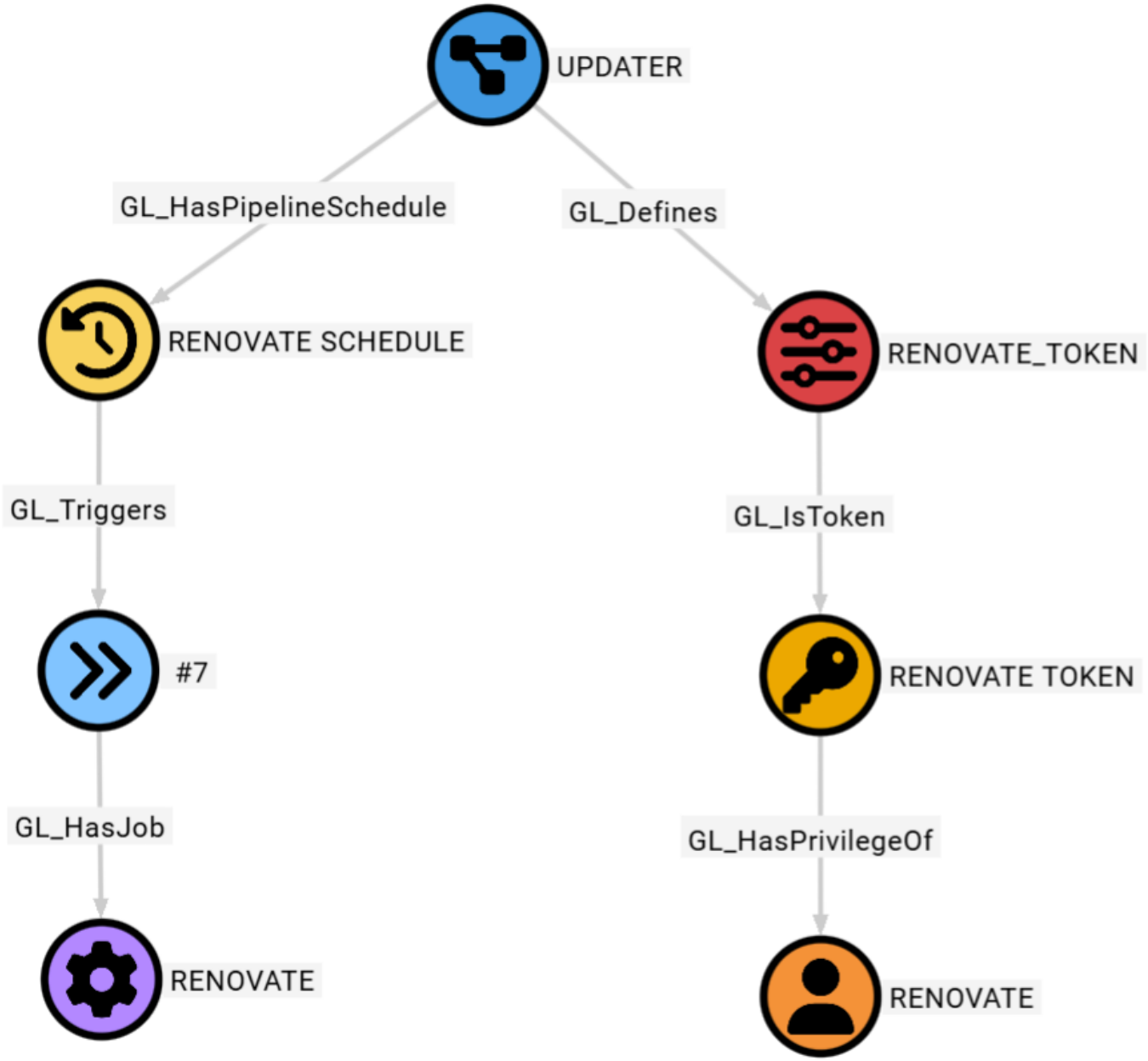
Attack path

1. Find Renovate configuration and corresponding bot user*
2. Invite update bot to project matching auto discovery filter
3. Let it update outdated dependencies
4. Exfiltrate its GitLab access token



Renovate update bot abuse

Edge composition



Graphing your own GitLab Instance

Example workflow

Different use cases

Auditing vs attacking

Auditing

- Full read-only permissions assumed
  - Complete information available through REST / GraphQL APIs
- Data quality: neither false-positives nor false-negatives desirable
- Single gathering phase possible
  - Complete in-memory graph representation

Attacking

- Limited by permissions and scope of available tokens
  - API restrictions apply, some information only available through web interface
- Best effort, some heuristics apply, false positives slightly more tolerable?
- Iterative collection desirable
  - BloodHound's natural upsert semantics
  - OpenGraph's current lack of post-processing

Graphing your own GitLab instance

Requirements and workflow

- GitLab personal access token with **read_api** scope
  - “Grants read access to the API, including all groups and projects, the container registry, and the package registry”
- BloodHound user and password with read-only role
- Workflow
  1. Register custom icons
  2. Collect GitLab data
  3. Post-process/enrich data
  4. (Visualize)
  5. …
  6. Profit

Graphing your own GitLab instance

Quickstart

```
# Set credentials
$ export GITLAB_URL=https://git.pwn.gift/
$ export GITLAB_TOKEN=glpat-..
$ export BLOODHOUND_USERNAME=api
$ export BLOODHOUND_PASSWORD=...

# Register custom node types
$ gitlabhound register

# Review and ingest resulting JSON file
$ gitlabhound collect > gitlab.json

# Initiate “post-processing”, create hybrid edges
$ gitlabhound enrich > gitlab-enrichment.json

# Get an idea of the data set
$ gitlabhound visualize gitlab.json gitlab-enrichment.json > schema.dot
```

Graphing your own GitLab instance

Hybrid edges

Active Directory

- Standard collection using SharpHound or AD Explorer snapshot
- On-premise runner mapping based on IP information available through ADIDNS data (objectclass=dnsnode)
  - Collect using TrustedSec ldapsearch BOF or pyldapsearch utility
  - Parse using Matt Creel's BOFHound
    - https://github.com/coffeegist/bofhound/pull/51

Entra ID / Azure

- AzureHound (best >= 2.10)
  - Includes Daniel Heinsen's support for federated identity credentials

Graphing your own GitLab instance

Secrets

GitLab

- Repository content/history
  - https://github.com/trufflesecurity/trufflehog
- Job logs and artifacts
  - https://github.com/CompassSecurity/pipeleek

```
# Map detected secrets to enumerated GitLab resources
$ GitLabHound enrich secrets pipeleek.json trufflehog.json > gitlab-secrets.json
```

Defensive Considerations

Hardening measures

Defensive Considerations

Hardening measures

Secret exposure

- Review and reduce effective permissions
- Use GitLab's built-in secret detection
- Include job logs and artifacts in periodic secret scans
- Define CI/CD variables as hidden and scope them to specific environments

Branch protection

- Implement strict branch protection rules, require:
  - Review from code owners
  - Multiple approvers

Pipeline abuse

- Review OIDC trust relationships and scope them explicitly as narrow as possible
- Avoid shared (self-hosted) runners and unsafe executors, harden the runtime environment
  - https://docs.gitlab.com/runner/security/

Defensive Consideration

Renovate

- Be aware of the Renovate security model
  - Code execution by renovated repositories within the bot context is assumed
  - All renovated repositories (by the same bot) must have the same security classification
- Disable **autodiscover** or set a strict **autodiscoverFilter** such that the Renovate bot can not be auto invited

https://docs.renovatebot.com/security-and-permissions/#security-awareness-for-self-hosted-renovate-instances

Identify security-relevant (mis)configurations

Saved queries

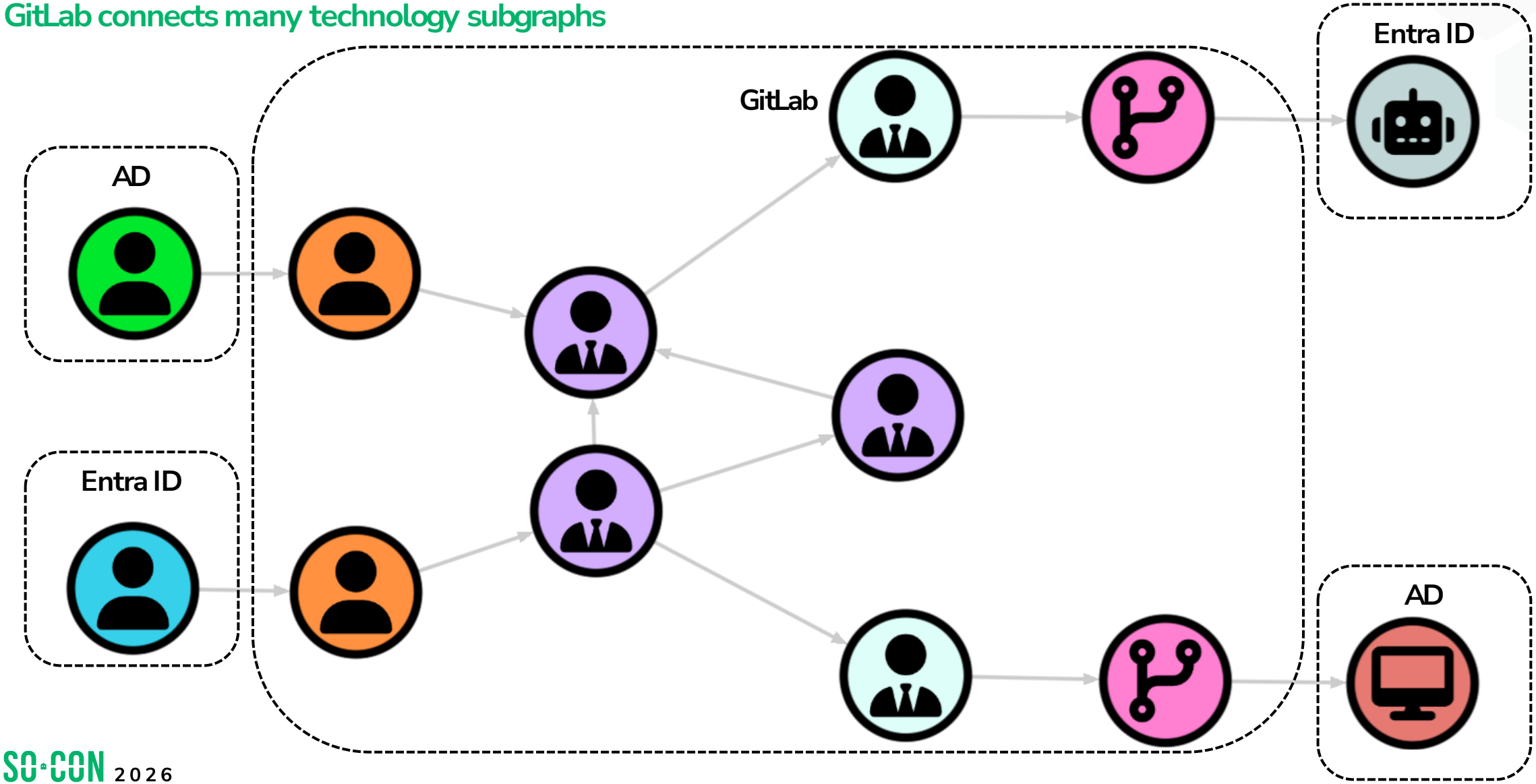
- List all instance administrators
- Display group hierarchy
- Privileged hybrid identities
- Runners with shell executor
- Detected secrets
- Unrestricted signup without admin approval
- Who has access to a given branch?
- Which branches can a given user push to?
- Users without a second authentication factor (within GitLab)
- …

Conclusion

Current state, future work?

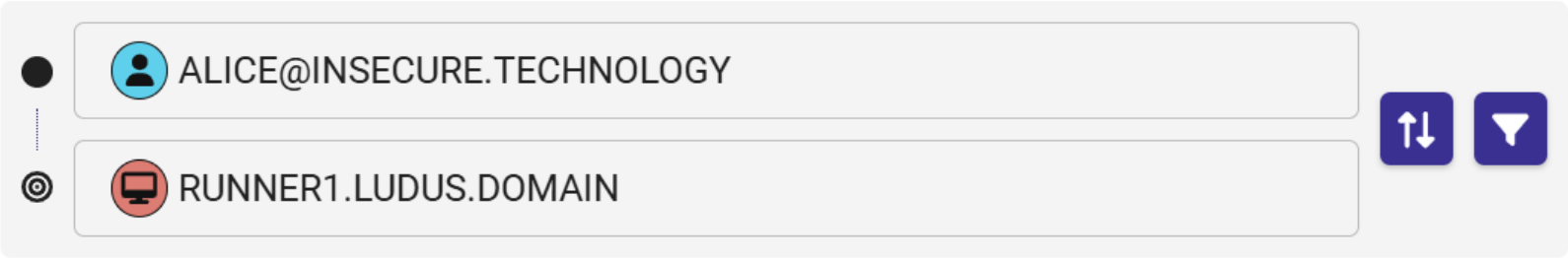
Conclusion

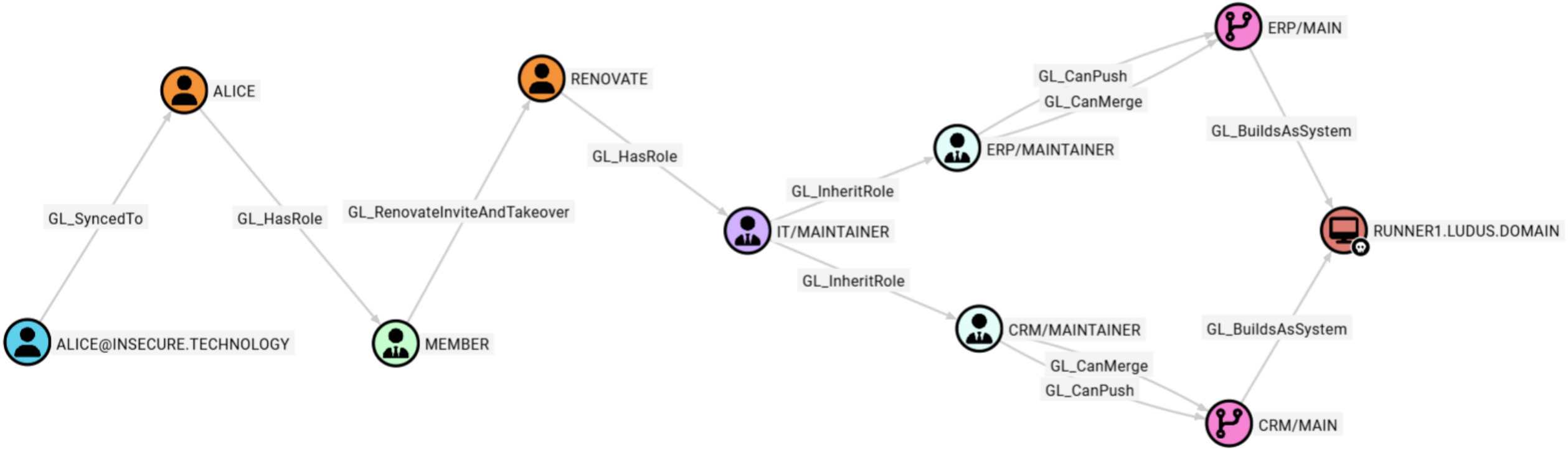
GitLab connects many technology subgraphs



Conclusion

GitLab connects many technology subgraphs





Conclusion

GitLabHound beta collector

- OpenGraph makes it trivial to ingest custom data into BloodHound
  - Modeling is the challenging part

- GitLab collector PoC written in Go: https://github.com/CompassSecurity/GitLabHound

  ~ 25 node types, ~50 edges
  - 4 hybrid edges:
    - LDAP / Entra ID authentication: **GL_SyncedTo**
    - Self-hosted on-premise runners: **GL_BuildsAsSystem**
    - Azure OIDC trust relationships: **GL_CanAssumeIdentity**
    - Secret detection: **GL_ContainsCredentialsFor***

- GitLab is complex, expect missing functionality and bugs, patches welcome

Reference

Tooling

- https://github.com/CompassSecurity/GitLabHound
- https://github.com/SpecterOps/GitHound
- https://github.com/CompassSecurity/pipeleek
- https://github.com/synacktiv/nord-stream
- https://github.com/praetorian-inc/glato
- https://github.com/h4wkst3r/SCMKit
- https://github.com/Frichetten/gitlab-runner-research

Reference

CI/CD Research

- The Illusion of Isolation: How Isolation Failures in CI/CD Servers Lead to RCE and Privacy Risks, Black Hat Asia 2025, Tian Zhou, Yiwen Wang, Xiu Zhang, Abstract, Slides, Talk
- Grand Theft Actions: Abusing Self-Hosted GitHub Runners at Scale, Adnan Khan and John Stawinski, DEF CON 32, Abstract, Slides , Talk
- Self-Hosted GitHub Runners: Continuous Integration, Continuous Destruction, Adnan Khan and John Stawinski, Black Hat USA 2024, Abstract, Slides, Talk
- The Dark Side of GitHub Actions, RomHack 2024, Adnan Khan, Slides, Talk
- Phantom of the Pipeline: Abusing Self-Hosted CI/CD Runners, ShmooCon 2023, Adnan Khan, Mason Davis, and Matt Jackoski, Abstract, Talk
- Securing the Pipeline: Protecting Self-Hosted GitHub Runners, BSides SF 2023, Adnan Khan, Slides, Talk
- Automating the extraction of secrets stored inside CI/CD systems, THCon 2023, Hugo Vincent, Théo Louis-Tisserand, Abstract, Slides Talk, Blog
- Controlling the Source: Abusing Source Code Management Systems, Brett Hawkins, Black Hat USA 2022, Abstract, Slides, Talk, Whitepaper, Blog

Thank you, questions?

Feedback